

27.1. La fonction `ecran()` — éteindre et rallumer l'écran

`#include <ecran>` : `ecran(0)` éteint l'écran, `ecran(1)` le rallume — le temps de dessiner beaucoup d'un coup.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `ecran(0)` éteint l'écran, `ecran(1)` le rallume. Écran éteint, on peut écrire dans toutes les cases sans attendre : c'est le moment d'un gros dessin, comme tout un niveau.

La ligne à écrire : `#include <ecran>`.

Son argument : 0 ou 1, écrit en clair.

Ce qu'elle coûte : environ **90 octets** avec la mise en place de l'écran ; un appel seul, quatre octets.

Essaie : enlève `ecran(1)` : l'écran reste blanc, le programme continue pourtant.

On retrouve `ecran` dans 31 leçons

[28](#), [29](#), [30](#), [31](#), [33](#), [67](#), [68](#), [69](#), [70](#), [71](#), [79](#), [80](#), [94](#), [95](#), [96](#), [97](#), [98](#), [99](#), [101](#), [102](#), [103](#), [105](#), [106](#), [113](#), [115](#), [119](#), [120](#), [124](#), [125](#), [126](#), [134](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **288 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	89 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
Diviser	17 o	← poser() — divise — le processeur ne sait pas le faire
Reste	10 o	← poser() — le reste d'une division
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	16 o	← CopierTuiles — 1 lettre sur 43

Lettres gravées : A

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 1	43 o sur place, dans main() + AdresseCase (10 o), Diviser (17 o), Reste (10 o)
ecran()	× 2	21 o sur place, dans main() + AttendreVBlank (16 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
poserS()	× 1	0 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

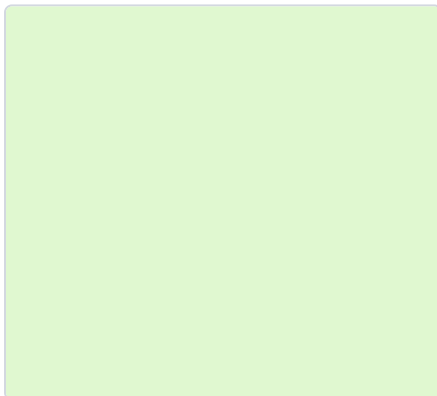
```
// ---- #include <ecran> : éteindre et rallumer l'écran ----  
// Écran éteint, on remplit trois lignes d'un coup, puis on le rallume.  
  
#include <ecran>      // écran() : éteindre (0), rallumer (1)  
#include <poserS>  
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

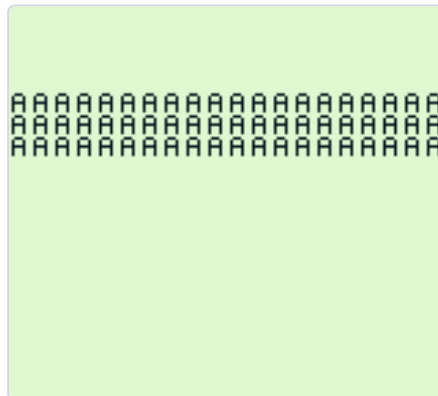
```
int main() {  
    écran(0);                // éteint : on écrit sans attendre  
    for (uint8_t i = 0; i < 60; i++) {  
        poserS(i, 4, ALPHABET[0]);    // 60 A : trois lignes pleines  
    }  
    écran(1);                // rallumé : tout apparaît d'un coup  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois lignes pleines de A, apparues d'un seul coup.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ l'écran est rallumé
- ✓ les trois lignes sont pleines

